Generative-AI–Based Automated Analysis and Risk Scoring of Fraudulent APKs

Datasets and Solution Approach

1 Introduction

Android remains the dominant mobile platform and, consequently, the primary target of mobile-banking fraud. Malicious APKs increasingly impersonate legitimate banking, payment, and utility applications to steal one-time passwords (OTPs), harvest credentials, abuse accessibility services, and exfiltrate sensitive data. Manual triage of such samples is slow and does not scale with the volume of submissions a fraud or Security Operations Centre (SOC) team receives.

This document describes the *datasets we will use* to train and validate the machine-learning component of our platform, and the *solution approach* we will follow to deliver automated, explainable risk assessment of suspicious APKs. Our system combines transparent rule-based scoring, supervised machine learning, controlled dynamic behaviour analysis, and a generative-AI reasoning layer that produces human-readable investigation reports.

2 Datasets

The machine-learning model will be trained, validated, and stress-tested on a combination of established Android-malware corpora and curated benign application sets. We deliberately combine *older, well-labelled* datasets (for breadth of malware families and reproducibility) with *recent* datasets and live sampling (to mitigate concept drift, since malware behaviour evolves over time).

2.1 Primary training and benchmark datasets

- **Drebin.** A widely cited Android-malware benchmark containing approximately **5,560 malicious** samples spanning **179 malware families**, alongside a large set of benign applications. Crucially, Drebin provides *pre-extracted static feature sets* per sample—requested and used permissions, API calls, intent actions, hardware components, application components, and embedded network indicators—together with a family-label mapping. These ready-made feature vectors allow us to train and benchmark classifiers without reprocessing raw APKs, making it our primary baseline dataset.
- **CICMalDroid 2020.** A modern dataset of roughly **17,000** Android samples organised into five classes: *Adware*, *Banking malware*, *SMS malware*, *Riskware*, and *Benign*. It includes both static attributes and dynamically observed behaviour (system calls, binder activity, composite behaviours). Its banking- and SMS-malware categories align directly with our fraud-triage objective, and its recency helps the model generalise beyond the older Drebin distribution.
- **CIC-AndMal2017.** A dataset emphasising *dynamic and network-traffic* features, comprising several hundred malware samples across major families (ransomware, scareware, SMS malware, adware) and several thousand benign applications. We will use it to enrich the feature space with runtime and network-flow characteristics that static analysis cannot capture.

- **AndroZoo.** A continuously growing repository of **millions** of real-world APKs collected from Google Play and numerous third-party markets, each annotated with multi-engine anti-virus detection counts and collection metadata. We will use AndroZoo as a source of *fresh, diverse* benign and malicious samples drawn from recent time windows, specifically to build an out-of-distribution evaluation set for measuring concept drift and real-world false-positive rates.
- **Android Malware Genome Project (MalGenome).** A historical reference set of **1,260** malware samples across **49 families**. Although dated, it provides a clean, well-documented baseline of classic malware behaviours that is useful for sanity-checking detectors and for family-level analysis.
- **KronoDroid.** A hybrid dataset that records *timestamped* samples with both static and dynamic features captured on real devices and emulators. Its temporal labelling makes it particularly valuable for time-aware evaluation, i.e. training on earlier samples and testing on later ones to simulate realistic deployment and quantify drift.

2.2 Benign application sources

A robust classifier requires a representative, modern benign distribution to keep false-positive rates low. In addition to the benign partitions of the datasets above, we will curate benign applications from:

- **F-Droid** — a catalogue of free and open-source Android applications, useful because the sources are auditable and the apps are demonstrably non-malicious.
- **Google Play (top and long-tail apps via AndroZoo)** — to capture the permission and API profiles of mainstream commercial applications, including advertisement- and analytics-heavy apps that are benign but superficially "suspicious."

2.3 Threat-intelligence corroboration source

For ground-truth verification and IOC enrichment (not as a training feature), we will consult **VirusTotal**, which aggregates verdicts from many anti-malware engines. This supports label validation and provides an independent second opinion during evaluation.

2.4 Summary of datasets

Dataset	Approx. scale	Feature type	Role in our work
Drebin	5.5k mal. + benign	Static (perms, API, intents, IOCs)	Primary baseline; pre-extracted features for fast training
CICMalDroid 2020	∼17k	Static + dynamic	Modern multi-class (incl. banking/SMS) training
CIC-AndMal2017	∼5.5k	Dynamic + network	Runtime/network feature enrichment
AndroZoo	Millions	Raw APKs + AV labels	Fresh sampling; drift / false-positive evaluation
MalGenome	1.26k	Static	Historical baseline; family sanity checks

Dataset	Approx. scale	Feature type	Role in our work
KronoDroid	Tens of thousands	Hybrid, timestamped	Time-aware (drift) evaluation
F-Droid / Play	Curated	Raw APKs	Modern benign distribution
VirusTotal	On-demand	Multi-engine verdicts	Label validation / IOC corroboration

Table 1: Datasets to be used for model training, benchmarking, and evaluation.

2.5 Data preparation and partitioning

- **Normalisation and labelling.** Sample identities will be tracked by cryptographic hash; labels will be reconciled across datasets and corroborated with multi-engine verdicts to reduce label noise.
- **Class balance.** Because benign samples vastly outnumber malicious ones, we will apply class weighting and/or balanced resampling to avoid a model biased toward the majority class.
- **Splitting strategy.** We will use stratified train/validation/test splits for in-distribution evaluation and, additionally, a *temporal split* (train on older samples, test on newer ones) to obtain an honest estimate of real-world performance under concept drift.
- **Leakage control.** Family-aware and time-aware splitting will be used to prevent near-duplicate samples from the same campaign appearing in both training and test sets, which would otherwise inflate reported accuracy.

3 Solution Approach

Our approach treats APK risk assessment as a layered pipeline. Each layer contributes independent, explainable evidence, and the layers are fused into a single, transparent risk score accompanied by a natural-language report. Generative AI is used to *reason over and communicate* the extracted evidence, never as the sole detector.

3.1 Stage 1 — Intake and identity

Each submitted APK will be hashed (e.g. SHA-256/MD5) for de-duplication and blacklisting, and basic metadata (package name, version, target/minimum SDK, file size) will be recorded. Hash-based identity enables consistent labelling and correlation with threat-intelligence feeds.

3.2 Stage 2 — Static analysis

Static analysis inspects the application without executing it and forms the backbone of fast, safe triage. We will extract:

- **Manifest and permissions:** declared components, intent filters, and the set of requested permissions, with emphasis on fraud-relevant permissions (SMS, contacts, accessibility, overlay, install-packages, phone state).

- **Suspicious API usage:** references to APIs associated with OTP theft, accessibility abuse, dynamic code loading, command execution, and network communication, weighted by how discriminative each is in benign versus malicious apps.
- **Indicators of compromise (IOCs):** URLs, IP addresses, domains, embedded tokens, and likely secrets, including indicators recovered by decoding obfuscated (e.g. Base64/hex) strings and by scanning bundled native libraries.
- **Obfuscation and packing signals:** identifier-renaming ratios, multiple DEX files, high-entropy blobs, dynamic loaders, and packer/compiler fingerprints.
- **Signing certificate and attack surface:** certificate red-flags (debug/placeholder identities, implausible validity) and exported components reachable by other applications.

Signature-based screening (rule sets expressing known fraud patterns) and a mapping of findings to a standard adversary-technique taxonomy (MITRE ATT&CK for Mobile) will be applied to make the output actionable for analysts.

3.3 Stage 3 — Controlled dynamic analysis

Because sophisticated samples conceal behaviour from static inspection (e.g. encrypted payloads decrypted only at runtime), we will optionally execute samples inside an *isolated, instrumented sandbox*. The sandbox will install the application, exercise all of its entry points (launcher activity, broadcast receivers, and background services), and observe runtime behaviour: granted permissions, started services, contacted network endpoints, file activity, and security-relevant API invocations.

To counter anti-analysis behaviour, the sandbox will apply *evasion-resistance* techniques—presenting realistic device characteristics so that emulator-detection checks do not suppress malicious behaviour—and will capture dynamically loaded code and command-and-control endpoints. All execution is confined to a disposable, isolated environment and is strictly opt-in; the static pipeline never executes the sample.

3.4 Stage 4 — Feature engineering

Static and dynamic observations will be transformed into a fixed-length feature representation for the classifier. The representation will be drawn from features that are expressed consistently across data sources, including:

- permission presence vectors;
- suspicious-API presence/frequency vectors;
- intent and component indicators;
- counts of network indicators and obfuscation signals;
- selected runtime-behaviour and network-flow features (when dynamic analysis is available).

We will favour features that are robust to superficial obfuscation and that remain comparable between training data and live submissions, in order to minimise feature drift between offline training and deployment.

3.5 Stage 5 — Machine-learning classification

We will train supervised classifiers to estimate the probability that a sample is malicious and, where data permits, its likely family/category (e.g. banking, SMS, adware). Candidate models include:

- **Gradient-boosted trees** (e.g. XGBoost/LightGBM) and **Random Forests**, chosen for strong tabular-data performance and interpretability through feature-importance analysis;
- **Logistic Regression** as a transparent baseline;
- Optional advanced directions, such as graph-based models over the application call graph, as a future extension.

The classifier output is treated as one *corroborating signal* fused with the rule-based score—not as a stand-alone verdict—so that the final assessment remains explainable.

3.6 Stage 6 — Hybrid risk scoring

The final risk score will combine the transparent rule-based score, the machine-learning probability, and threat-intelligence corroboration:

$$\text{Risk} = \text{RuleScore} + \text{ML adjustment} + \text{Intelligence corroboration},$$

normalised to a 0–100 scale and mapped to severity bands (Low, Medium, High, Critical). Every contribution is itemised so an analyst can see exactly *why* a sample received its score. Confirmed multi-engine detections raise confidence, while patterns characteristic of benign advertisement-heavy applications are down-weighted to control false positives.

3.7 Stage 7 — Generative-AI reasoning and reporting

A generative-AI layer will consume only the *extracted findings* (never the raw application) and produce: an executive summary for non-technical fraud managers, a technical narrative correlating the evidence into a likely attack chain, a list of indicators of compromise, and recommended actions for fraud/SOC teams. The model is prompted to reason critically about the evidence—distinguishing, for instance, benign library usage from genuine malicious intent—so that the report adds analytical value rather than merely restating the findings.

3.8 Stage 8 — Output and integration

The platform will present an analyst dashboard (risk gauge, itemised breakdown, evidence panels, and the AI report) and will export machine-readable artefacts: a formal PDF investigation report and IOC lists suitable for ingestion into blocklists and SOC tooling.

4 Evaluation Methodology

We will report standard classification metrics—precision, recall, F1-score, and ROC-AUC—computed on held-out data, with particular attention to:

- **False-positive rate on modern benign apps**, since over-flagging legitimate applications erodes analyst trust;
- **Temporal (drift) evaluation**, training on earlier samples and testing on later ones;
- **Cross-dataset generalisation**, training on one corpus and evaluating on another;
- **Explainability checks**, verifying that high-scoring decisions are supported by human-interpretable evidence.

5 Ethical and Safety Considerations

All potentially malicious samples will be handled and executed only within isolated environments using no personal accounts or networks. The system is designed for *defensive* purposes—first-level triage and investigation support for banking fraud and SOC teams. Reporting will be

faithful to the evidence, and limitations (e.g. concept drift, sandbox evasion, payloads gated behind unavailable infrastructure) will be stated transparently rather than overstated.